

Diagram Descriptions and Mermaid Source

The following diagrams can be rendered in a Mermaid-compatible editor or used as a guide for drawing diagrams in Word, draw.io, or Lucidchart.

1. Use Case Diagram

Explanation: The primary actor is the user. The user provides the input video and starts the application. The system processes frames, detects lanes, displays the annotated frames, and saves the output video. The user may stop the displayed result using Q or Esc.

```
flowchart LR
    User([User])
    Provide([Provide test_video.mp4])
    Start([Start lane detection])
    System[[Lane Detection System]]
    View([View live lane output])
    Stop([Stop with Q or Esc])
    Save([Save output_video.mp4])
    User --> Provide --> Start --> System
    System --> View
    User --> Stop --> System
    System --> Save
```

2. Workflow Diagram

Explanation: A frame is read, converted to grayscale, blurred, edge-detected, and masked to retain the road ROI. Hough Transform finds segments; the system separates, averages, and smooths left and right lane candidates. The result is then displayed and written to the output video. This repeats until processing ends or the user exits.

```
flowchart TD
    A([Start]) --> B[Open input video]
    B --> C{Frame available?}
    C -- No --> Z[Release resources and save output]
    C -- Yes --> D[Convert to grayscale]
    D --> E[Apply Gaussian blur]
    E --> F[Apply Canny edge detection]
    F --> G[Apply trapezium ROI mask]
    G --> H[Detect segments with Hough Transform]
    H --> I[Separate left and right by slope]
    I --> J[Average and smooth lane lines]
    J --> K[Draw lanes and FPS on original frame]
    K --> L[Display and write output frame]
    L --> M{Q or Esc pressed?}
    M -- No --> C
    M -- Yes --> Z
```

3. Sequence Diagram

Explanation: The user starts `main.py`, which opens the video through OpenCV. For every frame, `main.py` invokes the utility functions, receives averaged lane lines, overlays them, writes the output, and displays it. Finally, video resources are released.

```
sequenceDiagram
    actor User
    participant Main as main.py
    participant CV as OpenCV VideoCapture
    participant Utils as utils.py
    participant Writer as Output VideoWriter
    User->>Main: Run python src/main.py
    Main->>CV: Open test_video.mp4
    loop For each frame
        CV-->>Main: Return frame
        Main->>Utils: Detect edges, ROI, Hough lines
        Utils-->>Main: Averaged lane lines
        Main->>Main: Smooth and overlay lines/FPS
        Main->>Writer: Write annotated frame
    end
```

```

Main-->>User: Display output frame
end
Main-->>CV: Release video capture
Main-->>Writer: Release output video
Main-->>User: Show saved output location

```

4. System Architecture Diagram

Explanation: The input layer supplies the road video. The processing layer (`main.py` and `utils.py`) performs the detection pipeline. The output layer displays annotated frames and stores the processed MP4. OpenCV provides image and video operations, and NumPy supports array calculations.

```

flowchart LR
    Input[test_video.mp4\nInput Layer] --> Main[main.py\nApplication Controller]
    Main --> Utils[utils.py\nProcessing Functions]
    Utils --> P1[Grayscale + Gaussian Blur]
    P1 --> P2[Canny Edge Detection]
    P2 --> P3[ROI Masking]
    P3 --> P4[Hough Line Transform]
    P4 --> P5[Lane Averaging + Smoothing]
    P5 --> Main
    Main --> Display[Live OpenCV Window]
    Main --> Output[output_video.mp4\nOutput Layer]
    OpenCV[(OpenCV)] -. image/video operations .-> Main
    NumPy[(NumPy)] -. array calculations .-> Utils

```

5. Component Diagram

Explanation: `main.py` manages the video loop, line smoothing, display, FPS, and saving. `utils.py` contains reusable preprocessing and lane-detection functions. The structure separates responsibilities, making the code easier to test and explain.

```

flowchart TB
    Main[main.py\nVideo loop, smoothing, display, saving]
    Utils[utils.py\nPreprocessing, ROI, line detection, drawing]
    CV[OpenCV Library]
    NP[NumPy Library]
    In[(Input Video)]
    Out[(Output Video)]
    In --> Main
    Main <--> Utils
    Main --> Out
    Main --> CV
    Utils --> CV
    Utils --> NP

```